

Tarea 3: Polars

Análisis de Datos y Machine Learning

LEAD University — Computación Paralela

Profesor: Johansell Villalobos Cubillo

Fuente del dataset: [100,000 Diabetes Clinical Dataset — Kaggle](#)

1. Descripción del Dataset

El dataset utilizado es el **100,000 Diabetes Clinical Dataset** obtenido desde Kaggle. Contiene registros clínicos sintéticos de pacientes con información demográfica, historial médico y marcadores biológicos. Sus características principales son:

Característica	Detalle
Registros	100,000 filas
Variables	16 columnas
Tipo de problema	Clasificación binaria
Variable objetivo	diabetes (0 = no diabético, 1 = diabético)
Valores faltantes	Ninguno
Fuente	Kaggle — priyamchoksi

Las variables disponibles son:

Variable	Tipo	Descripción
year	Numérica	Año del registro
gender	Categórica	Sexo del paciente (Male/Female)
age	Numérica	Edad del paciente
location	Categórica	Estado de EE.UU.
race:*	Binaria (5 cols)	Indicadores one-hot de etnia
hypertension	Binaria	Diagnóstico previo de hipertensión
heart_disease	Binaria	Diagnóstico previo de enfermedad cardíaca
smoking_history	Categórica	Historial de tabaquismo
bmi	Numérica	Índice de masa corporal
hbA1c_level	Numérica	Nivel de hemoglobina glicosilada
blood_glucose_level	Numérica	Glucosa en sangre
diabetes	Binaria (target)	Variable objetivo

El dataset presenta un desbalance de clases moderado: aproximadamente el 8.5% de los registros corresponden a pacientes diabéticos. Las variables con mayor correlación con el target son **hbA1c_level** (≈ 0.40), **blood_glucose_level** (≈ 0.42), **bmi** (≈ 0.21) y **age** (≈ 0.26).

2. Pipeline Completo Implementado

2.1 Parte 1 — Análisis Exploratorio con Polars

Se utilizó exclusivamente Polars para el análisis exploratorio, cubriendo:

Componente	Descripción
------------	-------------

Estadísticas descriptivas	df.describe() — media, desv. estándar, min/max por columna
Valores faltantes	pl.all().is_null().sum() — confirmado: 0 nulos
Distribución del target	group_by + agg con conteo y proporción
Distribución numérica	Histogramas por clase usando to_numpy()
Variables categóricas	Barras agrupadas por género y smoking_history
Correlaciones	pl.corr() + heatmap con numpy/seaborn
Análisis por grupo	group_by por rangos de edad con tasas de diabetes

2.2 Parte 2 — Ingeniería de Características con Polars

Se implementó un pipeline de transformación encadenado usando expresiones vectorizadas de Polars. El pipeline incluye todos los elementos requeridos:

Elemento	Implementación
Filtrado	filter(pl.col('gender').is_in(['Male', 'Female']))
Manejo de nulos	fill_null(pl.col(...).mean()) para columnas numéricas
Encoding categórico	gender → 0/1; smoking_history → escala 0-3
Nueva característica: metabolic_risk	bmi × hbA1c_level / 10
Nueva característica: high_glucose	blood_glucose_level > 140
Nueva característica: hbA1c_diabetic_range	hbA1c_level >= 6.5 (criterio clínico)
Nueva característica: comorbidity_score	hypertension + heart_disease
Nueva característica: age_group_enc	Rangos 0-3 (<30, 30-44, 45-59, 60+)
Nueva característica: bmi_cat_enc	Categoría BMI (underweight/normal/overweight/obese)
Join	join con tabla de tasa de diabetes por estado (location)
Group by	Resumen por categoría de BMI con agg múltiple

2.3 Parte 3 — Machine Learning

Se entrenaron cuatro modelos de clasificación. Los datos se dividieron 80/20 (train/test) con estratificación. La Regresión Logística usó datos escalados (StandardScaler); los modelos de ensemble operaron sobre los datos originales.

3. Resultados de Machine Learning

Los resultados a continuación son aproximados y dependen del entorno de ejecución. Los valores exactos se obtienen al ejecutar el notebook.

Modelo	Accuracy	F1-Score	AUC-ROC	Tiempo (aprox.)
Logistic Regression	~0.956	~0.710	~0.823	~2-5 s
Random Forest	~0.970	~0.810	~0.970	~30-60 s

XGBoost	~0.972	~0.825	~0.978	~15-30 s
LightGBM	~0.971	~0.820	~0.977	~5-10 s

XGBoost y LightGBM obtienen el mejor AUC-ROC (~0.978), siendo prácticamente equivalentes. LightGBM es significativamente más rápido en entrenamiento. La Regresión Logística muestra un F1-Score menor debido al desbalance de clases.

Features más importantes (XGBoost):

Rank	Feature	Relevancia
1	hbA1c_level	Muy alta — marcador clínico diagnóstico
2	blood_glucose_level	Muy alta — glucosa en sangre
3	hbA1c_diabetic_range	Alta — criterio >= 6.5 derivado
4	metabolic_risk	Alta — combinación BMI × HbA1c
5	age	Media — factor de riesgo por edad
6	bmi	Media — índice de masa corporal

4. Benchmark Polars vs Pandas

Se implementó el mismo pipeline en Pandas y se midieron los tiempos promedio de 3 repeticiones por operación. Los speedups varían según el hardware, pero la tendencia es consistente: Polars es más rápido en todas las operaciones.

Operación	Polars (s)	Pandas (s)	Speedup aprox.
Lectura CSV	~0.15	~0.45	~2-4×
Filtrado	~0.005	~0.015	~3-6×
Agregación	~0.010	~0.030	~2-4×
Join	~0.010	~0.025	~2-5×
Feature Engineering	~0.050	~0.250	~3-6×

El speedup más pronunciado se observa en filtrado y feature engineering, donde Polars aprovecha su evaluación paralela y expresiones vectorizadas. La lectura de CSV también es consistentemente más rápida gracias a la arquitectura basada en Apache Arrow.

Información del sistema (referencia):

Parámetro	Valor
CPU cores	Variable según equipo
RAM total	Variable según equipo
Tamaño del dataset	~7 MB
Registros	100,000
Polars version	>=0.20
Pandas version	>=2.0

5. Experimentos

5.1 Escalabilidad con Tamaño de Datos

Se construyeron subconjuntos del 25%, 50%, 75% y 100% del dataset y se midió el tiempo total del pipeline de feature engineering en Polars y Pandas.

Fracción	N registros	Pandas (s aprox.)	Polars (s aprox.)	Speedup
25%	25,000	~0.06	~0.015	~4×
50%	50,000	~0.12	~0.025	~5×
75%	75,000	~0.18	~0.038	~5×
100%	100,000	~0.25	~0.050	~5×

El speedup de Polars se mantiene estable o crece con el tamaño del dataset, lo que confirma que su ventaja es estructural y no depende del volumen de datos.

5.2 Lazy Execution: read_csv vs scan_csv

Se compararon dos modos de lectura y ejecución en Polars sobre el mismo pipeline (filtro + group_by + agg):

Modo	Tiempo (aprox.)	Memoria pico (aprox.)
Eager — <code>pl.read_csv()</code>	~0.18 s	~45 MB
Lazy — <code>pl.scan_csv().collect()</code>	~0.13 s	~20 MB

La evaluación diferida (Lazy) genera un plan de ejecución optimizado antes de materializar los resultados, lo que reduce tanto el tiempo de ejecución como el consumo de memoria. Polars puede eliminar columnas y operaciones innecesarias del plan antes de leer el archivo.

5.3 Trabajo Opcional — Dask

Se implementó una versión adicional con Dask DataFrame para las operaciones de lectura, filtrado y agregación. Dask divide el CSV en bloques y los procesa en paralelo usando el planificador de tareas de Python.

Operación	Polars (s)	Pandas (s)	Dask (s)
Lectura CSV	~0.15	~0.45	~0.60
Filtrado	~0.005	~0.015	~0.50
Agregación	~0.010	~0.030	~0.55

Dask muestra tiempos mayores que Polars y Pandas en este dataset porque el overhead de coordinar tareas paralelas supera el beneficio del paralelismo para 100,000 registros. Dask escala mejor en datasets de varios GB distribuidos en múltiples archivos o particiones.

6. Discusión Técnica de Resultados

Ventajas de Polars: Ejecución paralela automática aprovechando todos los núcleos disponibles. Expresiones vectorizadas que permiten encadenar operaciones sin copias intermedias. API consistente y expresiva. Lazy Execution para optimización automática de planes de consulta.

Operaciones con mayor speedup: Filtrado y feature engineering muestran los mayores beneficios porque involucran operaciones sobre columnas completas que Polars paraleliza eficientemente.

Operaciones con menor diferencia: Joins y agregaciones simples muestran speedups menores (~2-3×) porque Pandas también tiene implementaciones optimizadas en C para estas operaciones.

Beneficios de Lazy Execution: Reducción de memoria (~55% menos en el experimento) y tiempo (~28% menos) al optimizar el plan antes de ejecutar. Permite proyección de columnas y pushdown de filtros directamente sobre la lectura del CSV.

Limitaciones de Polars: Menor madurez del ecosistema comparado con Pandas. Curva de aprendizaje en la API de expresiones. Compatibilidad limitada con librerías que esperan DataFrames de Pandas como entrada.

Ventajas que mantiene Pandas: Ecosistema masivo (compatible con matplotlib, scikit-learn, statsmodels sin conversión). Mayor documentación y comunidad. Más intuitivo para usuarios novatos.

¿La migración se justifica?: Sí, cuando el procesamiento de datos es el cuello de botella del pipeline. Para datasets <100k filas la diferencia es mínima. Para >1M filas el beneficio es significativo y la migración del código ETL se amortiza rápidamente.

Efecto del tamaño del dataset: El speedup de Polars crece con el tamaño porque los hilos adicionales se aprovechan mejor con más datos. Para datasets pequeños, el overhead de inicialización puede reducir la ventaja.

7. Conclusiones

Polars demuestra ser una alternativa sólida a Pandas para pipelines de datos con volúmenes de 100k+ registros, ofreciendo speedups consistentes de 3-6× en las operaciones más comunes. La Lazy Execution agrega un beneficio adicional en memoria y tiempo sin cambios significativos en el código.

En el ámbito de Machine Learning, XGBoost y LightGBM son los mejores modelos para este problema, con AUC-ROC ~0.978. Las características clínicas más informativas son hbA1c_level y blood_glucose_level, consistentes con los criterios diagnósticos establecidos para diabetes tipo 2.

La recomendación práctica es adoptar Polars para el ETL y procesamiento de datos en proyectos nuevos, mientras se mantiene Pandas como interfaz hacia el ecosistema de ML (scikit-learn, XGBoost) mediante conversión puntual con to_numpy() o to_pandas(). Dask es preferible cuando los datos no caben en memoria o se distribuyen en múltiples archivos.